

Esercitazione 3 - Puntatori e Allocazione Dinamica in C



In questa esercitazione occorre implementare tutte le funzioni definite nel file `store.h`.

Si ricorda che non si possono usare indici, ma occorre sfruttare l'aritmetica dei puntatori.

Il file `testing.h` è una libreria necessaria per i test, non dovete toccarla.

Questo progetto è compatibile sia con l'IDE CLion che con l'IDE Code::Blocks

Per usare questo progetto offline, naturalmente dovete prima scaricarlo.

Se usate CLion, aprite il file `Makefile` con Clion (Apri con -> Clion)

Se usate Code::Blocks, aprite il file con estensione `.cbp`

L'unico file che verrà corretto è il file `store.h`

Le funzioni da implementare sono di 5 tipi su 2 entità:

- costruzione di entità (ovvero la creazione di entità a partire da certi parametri)
- clonazione di entità (ovvero la creazione di una nuova entità a partire da un'altra)
- distruzione di entità (ovvero la deallocazione completa di una entità)
- serializzazione di entità (ovvero il salvataggio su file seguendo un certo formato)
- deserializzazione di entità (ovvero il caricamento da file formattato secondo un certo formato)

Qui c'è l'elenco completo dei test:

```
createCar(...)
  createCar(NULL, -1)
  createCar("Testing", -1)
  createCar("", 0)
  createCar("Testing", 0)
  createCar("Testing", 10)
  createCar("Testing", 10) - allocation
  createCar("Testing", 10) - name
  createCar("Testing", 10) - name (allocation)
  createCar("Testing", 10) - price

cloneCar(...)
  cloneCar(NULL)
  cloneCar(car)
  cloneCar(car) - allocation
  cloneCar(car) - name
  cloneCar(car) - name (allocation)
  cloneCar(car) - price

destroyCar(...)
  destroyCar(NULL)
  destroyCar(car)
  destroyCar(car) - de-allocation

serializeCar(...)
  serializeCar(NULL, NULL)
  serializeCar(car, NULL)
  serializeCar(NULL, fp)
  serializeCar(car, fp)
  serializeCar(car, fp) - file contents

deserializeCar(...)
  deserializeCar(NULL)
  deserializeCar(fp) - Test file 1
  deserializeCar(fp) - Test file 2
  deserializeCar(fp) - Test file 3
  deserializeCar(fp) - Test file 3 - allocation
  deserializeCar(fp) - Test file 3 - name
  deserializeCar(fp) - Test file 3 - name - allocation
  deserializeCar(fp) - Test file 3 - price
```

```
createStore(...)
  createStore(NULL, NULL, NULL, -1)
  createStore("", NULL, NULL, -1)
  createStore("Testing", NULL, NULL, -1)
  createStore("Testing", "Tester Manager", NULL, -1)
  createStore(NULL, NULL, cars, -1)
  createStore(NULL, NULL, cars, 0)
  createStore(NULL, NULL, cars, 1)
  createStore("", NULL, cars, -1)
  createStore(NULL, "", cars, -1)
  createStore("Testing Store", "Testing Manager", cars, 1)
  createStore("Testing Store", "Testing Manager", cars, 1) - allocation
  createStore("Testing Store", "Testing Manager", cars, 1) - name
  createStore("Testing Store", "Testing Manager", cars, 1) - name - allocation
  createStore("Testing Store", "Testing Manager", cars, 1) - manager name
  createStore("Testing Store", "Testing Manager", cars, 1) - manager name -
allocation
  createStore("Testing Store", "Testing Manager", cars, 1) - cars_count
  createStore("Testing Store", "Testing Manager", cars, 1) - cars[0] check

cloneStore(...)
  cloneStore(NULL)
  cloneStore(store)
  cloneStore(store) - allocation
  cloneStore(store) - name - allocation
  cloneStore(store) - cars - allocation
  cloneStore(store) - name check
  cloneStore(store) - cars_count check
  cloneStore(store) - cars[0] check

destroyStore(...)
  destroyStore(NULL)
  destroyStore(store)
  destroyStore(store) - de-allocation

serializeStore(...)
  serializeStore(NULL, NULL)
  serializeStore(store, NULL)
  serializeStore(NULL, fp)
  serializeStore(store, fp)
  serializeStore(store, fp) - file contents

deserializeStore(...)
  deserializeStore(NULL)
  deserializeStore(fp) - Test file 1
  deserializeStore(fp) - Test file 2
  deserializeStore(fp) - Test file 3
  deserializeStore(fp) - Test file 4
  deserializeStore(fp) - Test file 4 - allocation
  deserializeStore(fp) - Test file 4 - name - allocation
  deserializeStore(fp) - Test file 4 - manager name - allocation
  deserializeStore(fp) - Test file 4 - contents
```

FAQ

Non vedo tutti i test, come mai?

I test compariranno man mano che implementate correttamente le funzioni.

Quale è il formato dei file per la serializzazione/deserializzazione?

Avete dei file di esempio sia per `Car` che per `Store`

Posso modificare altri file a parte `store.h`?

No